HABIB UNIVERSITY

Data Structures & Algorithms

CS/CE 102/171 Spring 2025

Instructor: Maria Samad

Quiz # 1 Solution

QUESTION 1: (20 marks)

You are given an input string called ***str1*** containing 'n' characters including blank spaces, where 'n' is at least 2, and another string called ***word*** containing no spaces with length < n. You are also given a stack called ***StackChars*** of size 'n', which is initially empty. The program removes the specified word from the stack, while keeping all the other words in the same order of their appearance in string. Assume 'str1' and 'word' are correctly given to you with the allowed length, so you may use them directly in your algorithm, without having to do any error checking or taking input. However, do declare them in the algorithm to define what they are, including the stacks and/or other ADTs that you may use for your program.

DO NOT TRY TO ACHIEVE THE TASK IN ANY WAY OTHER THAN THE FOLLOWING STEPS:

1. It reads each character of string, ***str1*** and adds them to the stack, ***StackChars*** one at a time, such that the last letter of the string appears at the top of the stack.
2. Once all letters are in the stack, you CANNOT use ***str1*** again anywhere in the program. If it is being used for any reason – simple or complex, the remaining algorithm will NOT be graded
3. Using the stack of letters, i.e. ***StackChars***, you should update the same stack such that after removal of instances of ***word***, the letters in the stack appear in the same order as they were input in Step 1.

For example,

- Assume ***str1*** = "**I am not sad not bot"**, and given word to be removed = **"not"**, so as a result of first task, ***StackChars*** = ['I', ' ', 'a', 'm', ' ', 'n', 'o', 't', ' ', 's', 'a', 'd', ' ', 'n', 'o', 't', ' ', 'b', 'o', 't']
- Once ***StackChars*** is filled, remove the appearance of all **"not"** from the stack with stack resulting in: ***StackChars*** = ['I', ' ', 'a', 'm', ' ', ' ', 's', 'a', 'd', ' ', ' ', 'b', 'o', 't']

You must implement the above tasks by writing an algorithm for each part.

Restrictions:

- Do not **HARD CODE** your design for the given example. The algorithm should work for any valid sized string.
- Assume the length of string is 'n', where n > 1, so there will always be at least two letters in any given string, resulting in Stack of size at least 2
- Assume None represents empty slots in all the data structures used for this question
- You may use additional variables, but as minimum as possible
- You are **NOT** allowed to use **any** additional **ADT**, including ListADT
- **DO NOT USE ANY OTHER DATA STRUCTURES INCLUDING ListADT/Python lists apart from the ones permitted to use**
- **REMEMBER: Stacks and Queues are non-iterable and non-traversing data structures, so you CANNOT and MUST NOT do indexing within the data structures to access the elements**

SOLUTION:

#Required variable declarations and initialization

- str1 – string input containing 'n' characters, including blank spaces
- n – an integer representing the length of the string as well as the size of the stack, i.e., StackChars
- word – a string containing no spaces, representing the word to be removed from the input string, i.e., str1
- StackChars = [None] * n
- word_extra – an additional string variable to extract the word from StackChars and compare with given word
- str_extra – an additional string variable to store the string popped from the stack during the process

#Adding all the characters from the given string into the input stack, i.e., StackChars

- for each character in string, do the following:
    - push character in StackChars

- word_len = length of the given word #integer to run the loop for each character in word and compare with #each character popped from StackChars

#for all the characters in StackChars, check each word to match with the given word, so that it can be removed #from the stack, and we are left with the remaining phrase in Stack

- while StackChars is not empty, do the following:
    - character1 = top of the StackChars #get top character from stack to compare it with word character

      #if the character on the top of the stack is the same as the word's last letter, there might be a chance that #the word present on top of the stack is the same as the given word. As stacks are non-iterable, each #character must be removed one at a time and combined to form a word so that it can be compared. For #that we use an additional string variable called word_extra to get the concatenated word extracted from #the stack. If word_extra is the same as given word, it should be removed from the stack
    - if character1 == word[–1], then do the following:
        - for i in range(word_len – 1, –1, –1):
            - character2 = top of the StackChars
            - if character2 == word[i], then do the following:
                - character2 = pop from StackChars
                - word_extra = character2 + word_extra

          #if the extracted word is not the same as the given word, then these characters need not be #removed from the stack. In order to not lose the original data, we have an additional string #called str_extra that will keep on concatenating the updated string, so that we may add it back to #StackChars at the end
        - if word_extra != word, then do the following:
            - str_extra += word_extra
            - while (top of StackChars != blank space), do the following:
                - character2 = pop from StackChars
                - str_extra = character2 + str_extra

          #Extract blank spaces between words, & add to str_extra to keep the phrase in its correct form
        - character2 = pop from StackChars
        - str_extra = character2 + str_extra
        - word_extra = " " #Reset word_extra to an empty string for next usage

      #if the character is not the last letter of the word, then simply pop it from Stack and add it to str_extra
    - else if character1 != word[–1], then do the following:
        - character2 = pop from StackChars
        - str_extra = character2 + str_extra

#Add the updated string back into StackChars, as required by the algorithm

- for each character in str_extra, do the following:
    - push character on StackChars

QUESTION 2: (5 marks)

We have the following 2D-array stored in memory: ***arr1*** = [[1, 2, 3, 4, 5], [6, 19, 20, 21, 22], [11, 12, 7, 9, 11]]. Assume each number is 16 bits in size, and one location of memory is 8-bits wide. Calculate the address of the element, arr1[1][2]

SHOW ALL THE NECESSARY STEPS/FORMULA TO ANSWER THE ABOVE QUESTION, OTHERWISE, NO MARKS WILL BE AWARDED EVEN FOR CORRECT ANSWERS!

SOLUTION:

We know that for two-dimensional arrays, the address of each location is calculated as:

$$Memory\ Address = Base\_Address + (((i * number\ of\ columns) + j) * element\_size)$$

- Assume base address = 0
- i is the row index, which is given as 1
- j is the column index, which is given as 2
- number of columns from the array can be deduced = 5
- What about element size?
    - It needs to be calculated based on the size of the array value, as well as the memory location size, i.e.
        - element size = 16/8 = 2
- Therefore, memory address = 0 + ((1 * 5) + 2) * 2 = 0 + (5 + 2) * 2 = 0 + 7 * 2 = 0 + 14 = 14